

RIDEBURY

Evidence of original development

Submitted in response to the App Review decision of 26 August 2026 under Guideline 4.3(a)
— Design: Spam.

Submission ID	5dcc697c-137e-4dcc-bc9a-22f3832d339f
Review date	26 August 2026
Version reviewed	1.0 (4)
Review device	iPad Air 11-inch (M3)
App name	Ridebury (submitted previously as “Bus Jam Traffic Rush”)
Bundle ID	com.intakeentertainment.busjamtrafficrush
Developer	Intake Entertainment
Engine	Unity 6000.4.9f1, Universal Render Pipeline
Repository	github.com/oguztarhan/project_kocasinan (private)

Prepared 27 August 2026 by Oğuz Tarhan, Intake Entertainment.
All figures in this document were read directly from the project’s git repository and working tree on the date above.

Why we are sending this

Our submission was rejected with the note that the app “shares a similar binary, metadata, and/or concept as apps submitted to the App Store by other developers, with only minor differences,” and the review listed purchased or repackaged templates among the usual causes.

We are a two-person studio in Kayseri, Turkey. We did not buy a template, we did not use an app generator, and we did not start from anyone else’s source code. The project was started from an empty Unity project on 3 June 2026 and has been built by the two of us since then, in the open, one commit at a time. This document is the record of that work.

We want to be straightforward about one thing, because we think it is the heart of the matter. The *genre* is not ours: a queue of passengers boarding colour-matched vehicles is a well-established casual-puzzle format, in the same way that match-3 or endless-runner are formats. We are not claiming to have invented it. What we are claiming, and what the following pages document, is that the binary is not a repackage of anyone else’s: the code, the level system, the economy, the UI and — since the rejection — every character and vehicle model in the game were made by us.

What is in this document

- The full commit history: 144 commits across 42 working days by two named developers, from the first empty project to the current build.
- The source inventory: 89 C# files, 19,040 lines, and how that code grew week by week.
- Our Blender project files and render output for the 20 characters and 27 vehicles that are now in the game, plus the Python generator we wrote to build the vehicle fleet.
- Screenshots taken from the current build.
- An honest list of the third-party components we do use, and what each one is for.
- The changes we made after the 26 August review.

We are happy to give the review team read access to the private repository, or to walk through any part of it on a call. Contact details are on the last page.

Oğuz Tarhan
Intake Entertainment
27 August 2026

1. The project in numbers

Everything below was read from the repository on 27 August 2026.

First commit	3 June 2026, 22:46 — 680e1db “Initial commit”
Most recent commit	27 August 2026, 22:57 — e0339d8
Commits	144
Days with at least one commit	42
Contributors	2 people (3 git identities)
Tracked files in the repository	1,483
C# source files	89
C# lines in the shipping project	19,040
C# written and rewritten over the project's life	+34,715 / -8,801 lines
Largest single file	RideburyGame.cs — 5,056 lines
In-house 3D character models	20 (.fbx, authored in Blender)
In-house 3D vehicle models	27 (.glb, authored in Blender)
Interface languages	9
Sound assets	52

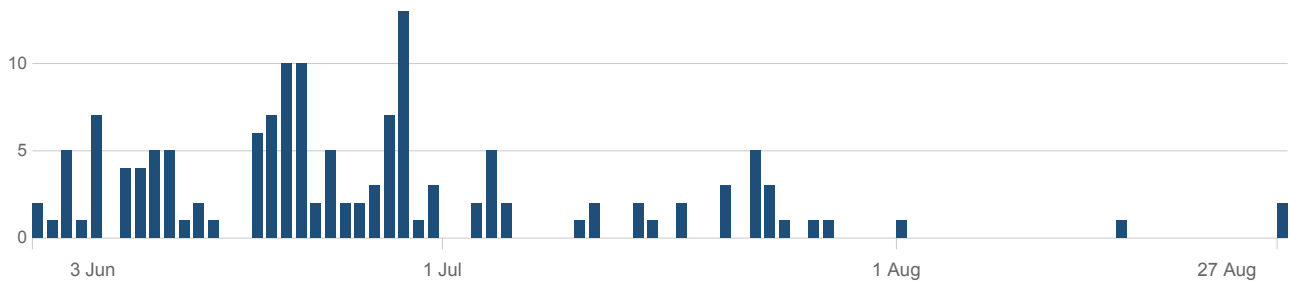
Who wrote it

Developer	Git identity	Commits	C# lines added / removed
Oğuz Tarhan	oguztarhan0@gmail.com	21	+11,288 / -2,020
Oğuz Tarhan	141235390+oguztarhan@users.noreply.github.com	49	+9,012 / -3,057
Kenan Kanter	burakka01 <kenankanter@gmail.com>	76	+14,415 / -3,724

The two Oğuz Tarhan rows are the same person committing from two machines (a GitHub web identity and a local one). No other identity has ever committed to this repository.

2. Development timeline

This is what the work actually looked like: a dense first month building the core game, a second month of balancing, store integration and platform work, then the release builds. Each bar is one calendar day.



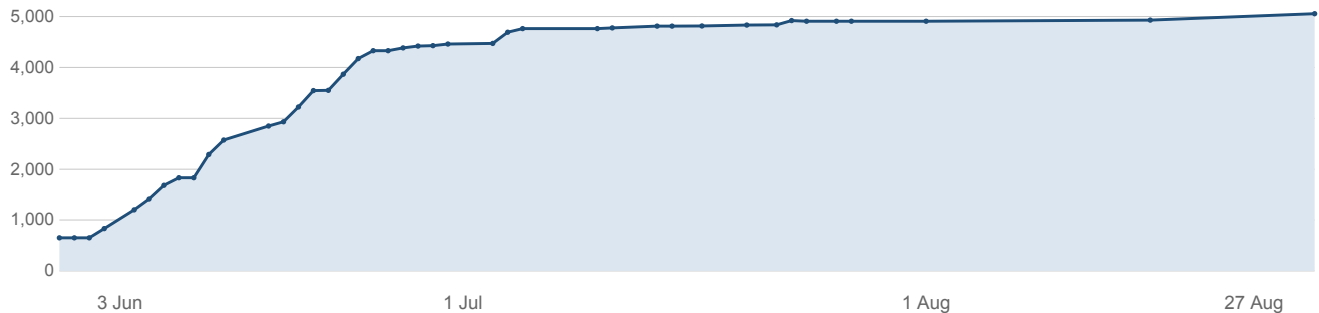
Commits per day, 3 June – 27 August 2026. Tallest day is 28 June, with 13 commits.

Milestones, in our own commit messages

Date	Commit	What happened
3 Jun	680e1db	Initial commit — empty Unity project
4 Jun	ef014dc	First playable loop; the main controller is 650 lines
7 Jun	b5448f6	“bus grid fixed”, then “grid system fixed” (e44c358) — the jam grid starts working
9 Jun	9415da1	“big patch major patch” — controller passes 1,000 lines
12 Jun	e9746f4	Menus, panels, shop and HUD baked; joker inventory; 9-language selector; social links
13 Jun	57d1f81	Levels tab added
20 Jun	b624c8b	First device build (“first build”)
21 Jun	c4e5bc6	Save system rewritten, then “MECHANIC UPDATE” (b24499d) and frame-rate work
23 Jun	0d101b7	Unified diagonal exit route (no mesh, side-lane routing) + 60 fps mobile pass
24 Jun	c26504f	New mechanic update
25 Jun	0145b27	Third vehicle type (minivan) and the vehicle wardrobe
26 Jun	ab6c059	Garage: hierarchy-bakeable panels, sizing / performance / visibility fixes
27 Jun	d8a8e02	Vehicle recolour system rebuilt (body + controlled dark glass); craft-from-shards economy; dynamic frame rate
28 Jun	c60fee3	Tutorial: tap-to-advance coach marks, with guards so it never repeats
4 Jul	42d8f1b	Chest tiers read by rope colour on a shared body; uniform card background
22 Jul	db5b068	SDK integration fixed
23 Jul	f36b3d5	“final push for ios and android”
27 Jul	819bb82	iOS release 1.1.2 (3) — production AdMob IDs, Firebase iOS config, audio-session fix, signing
16 Aug	68607a3	Post-release fix
27 Aug	c0eafcb	4.3(a) work — unused stock assets purged (2,672 files, -2.5M lines of asset YAML)
27 Aug	e0339d8	4.3(a) work — in-house characters and vehicles in, rename to Ridebury

3. The source code

A purchased template arrives finished. Ours did not — it was written in front of the commit history. The chart below tracks a single file, the main game controller, from the 650 lines it had on 4 June to the 5,056 lines it has today. Each dot is a commit day.



Lines in the main game controller (`BusJamGame.cs`, renamed to `RideburyGame.cs` on 27 August). Measured with `git show <commit>:<path>` for every commit in the history.

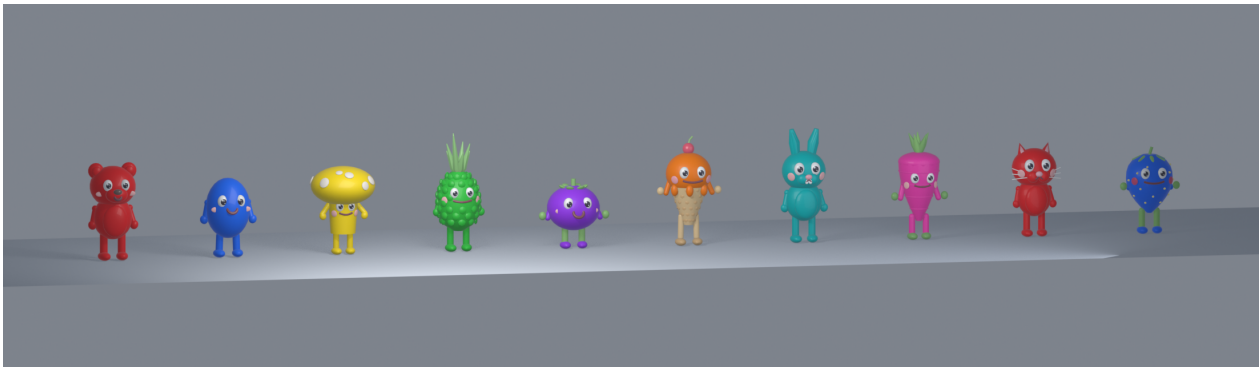
What the 89 files do

File	Lines	Responsibility
<code>RideburyGame.cs</code>	5,056	Core loop: jam grid, vehicle routing, boarding, scoring, level flow
<code>GameUI.cs</code>	1,505	HUD, panels, transitions (plus 6 partial-class files)
<code>MenuBackground.cs</code>	916	The animated main-menu scene, built procedurally at runtime
<code>GameUI.Garage.cs</code>	813	Garage, chests, crafting, shard economy
<code>LevelGenerator.cs</code>	773	Procedural level generation with a guaranteed-solvable ordering
<code>BootSplash.cs</code>	602	Studio splash and boot sequence
<code>RateUs.cs</code>	450	In-app review prompt logic
<code>LowPolyBuilder.cs</code>	418	Runtime mesh construction for city props
<code>AdManager.cs</code>	387	AdMob banner / interstitial / rewarded integration
<code>SaveSystem.cs</code>	363	Persistence and migration
<code>NotificationService.cs</code>	360	Local re-engagement notifications and channel management
<code>Sfx.cs</code>	340	Sound mixing and pooling
<code>Loc.cs</code>	335	Localisation table, 9 languages, live language switching
<code>GameUI.Vehicles.cs</code>	295	Vehicle wardrobe and collection screen
<code>LevelSelect.cs</code>	252	Level map and progression
<code>TutorialCoach.cs</code>	210	First-run coaching
<code>IAPManager.cs</code>	192	Unity IAP purchase flow
<code>ModelPool.cs</code>	182	Object pooling for vehicles and passengers
<code>VehiclePreview.cs</code>	170	Renders garage thumbnails from the 3D models at runtime
Editor/ (11 files)	—	Our own editor tooling: character importer, vehicle catalogue builders, garage UI baker, UI atlas baker, graphics-tier menu

The eleven files in `Assets/Scripts/Editor/` are custom Unity editor tools we wrote for this project — menu items such as *Ridebury › Build Vehicle Catalog* and *Ridebury › Build Vehicle Sets*. They exist because we author the content; a repackaged app has nothing to author.

4. The characters, modelled by us in Blender

Until this week the passengers waiting in the queue were a free Unity Asset Store character pack. That pack is used by many games, and we accept that it was the single strongest reason our build looked like other builds. We removed it. Over 26–27 August we modelled 20 replacement characters ourselves in Blender — fruit, vegetables and animals, each with its own silhouette, face and idle pose — and they are what ships now.



Ten of the twenty passengers, rendered from `blender/bus_queue_characters.blend`. The `.blend` file is in the repository.



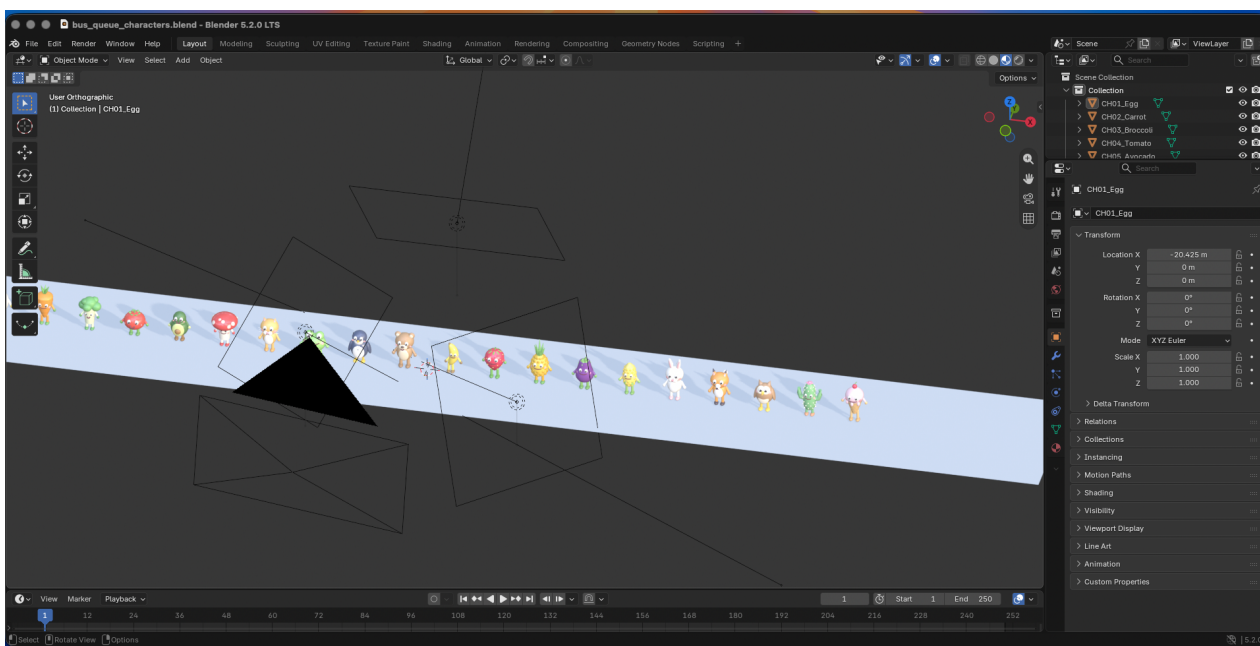
Close-up pass. Every model is built from our own primitives and materials — there is no purchased mesh and no scanned or generated asset in this set.

The Blender project, open on our machine

We work the way any small team works: model, render a contact sheet, look at it at game size, fix what does not read, repeat.



Expression tests for the bear, one of the twenty. We iterated on eyes, blush and mouth until the faces read at the size the game actually draws them (roughly 60 px tall on a phone).



bus_queue_characters.blend in Blender 5.2 LTS on 27 August 2026, with the twenty characters laid out on the queue strip. The outliner shows our naming scheme (CH01_Egg, CH02_Carrot, CH03_Broccoli, CH04_Tomato, CH05_Avocado...), which matches the twenty .fbx files in Assets/Characters/RideburyPeople/.

Both .blend files — bus_queue_characters.blend (2.2 MB) and vehicles.blend (3.8 MB) — are committed to the repository along with 18 render sheets from the sessions that produced them. If it would help, we can send the .blend files directly; they open in any Blender install and contain the full construction history.

5. The vehicles, generated by our own tool

The garage collection used to be Asset Store car packs. On 27 August we replaced 27 of the 28 entries with vehicles we generate ourselves. Rather than model each one by hand, we wrote a parametric generator in Python for Blender: a set of archetype profile curves (roof line, sill, belt line, glass bands) is lofted along the length of the body, wheel arches are cut with booleans, and each model gets our own chrome badge. 673 lines of generator code produce the whole fleet.



Ten of the generated cars. Each is a different archetype — hatchback, pickup, coupe, sedan, wagon — from the same loft engine, not a recoloured copy.



The bus and minivan sets from the same generator. 27 models in total, exported as .glb: 10 cars, 8 minivans and 9 buses.

The generator, in our repository

This is the head of `vehlib.py`, one of the two files. It is included in full in the evidence folder alongside `fleet.py`.

```
# Realistic-ish low-poly vehicle generator for project_kocasinan.
# One loft engine drives cars, vans and buses.  xf = 0.0 REAR .. 1.0 FRONT.
# Every model carries an ORIGINAL badge (chrome "K" disc). No real-world marks.
import bpy, bmesh, math

K = 16
J_GLASS_SIDE = {4, 5, 10, 11}      # verts per cross-section ring
J_TOP         = {6, 7, 8, 9}       # quad bands forming the side glass
J_UNDER       = {0, 15}            # roof / windshield / backlight bands
J_UNDER       = {0, 15}            # underbody

# ----- curves (PCHIP: smooth, no overshoot) -----
def pchip(xs, ys):
    n = len(xs)
    if n == 1: return lambda t: ys[0]
    h = [xs[i+1]-xs[i] for i in range(n-1)]
    d = [(ys[i+1]-ys[i])/h[i] if h[i] else 0.0 for i in range(n-1)]
    m = [0.0]*n
    m[0], m[-1] = d[0], d[-1]
    for i in range(1, n-1):
        if d[i-1]*d[i] <= 0: m[i] = 0.0
        else:
            w1, w2 = 2*h[i]+h[i-1], h[i]+2*h[i-1]
            m[i] = (w1*d[i-1] + w2*d[i]) / (w1+d[i-1] + w2+d[i])
```

`vehlib.py`, lines 1–28 of 462. With `fleet.py` (211 lines) this is the whole fleet.

What this changed in the build

	Before 27 Aug	After
Passenger characters	Free Asset Store pack (68 files)	20 models made by us in Blender
Garage vehicles	Asset Store car packs, 35 MB	27 generated .glb models, 10 MB
Vehicle art shipped in the binary	Third-party meshes	One retained third-party van, at our choice
Unused stock assets in the repo	2,672 files	Removed

One vehicle — the model we call “Connect” — is still a licensed Asset Store mesh. We kept it deliberately because it is the vehicle in our tutorial and our store screenshots. It is listed in the disclosure in section 7.

6. The current build

These were captured from the build in Unity on 27 August 2026, after the changes described above. The game is portrait; the review device was an iPad, where it runs in the same portrait layout.

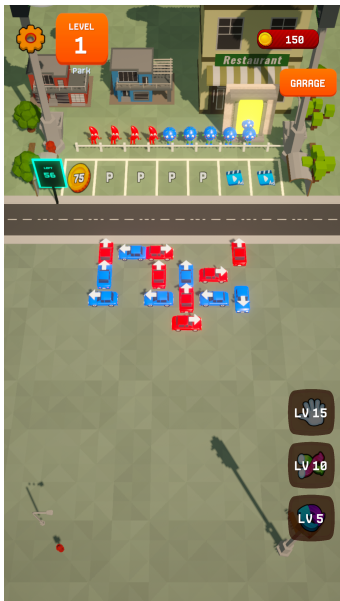
The interface ships in 9 languages — Turkish, English, German, Italian, Spanish, Chinese, French, Portuguese and Indonesian — switchable at runtime without a restart (Loc .cs, 335 lines). These captures are in English.



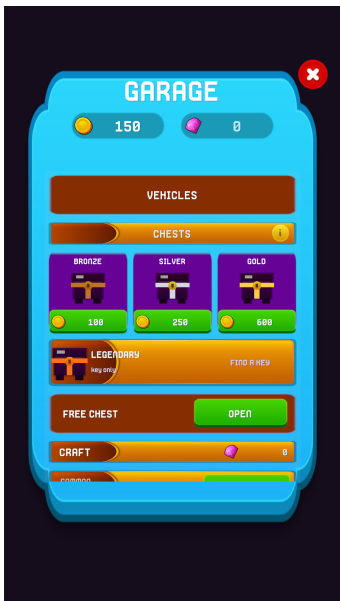
Studio splash. The logo is ours.



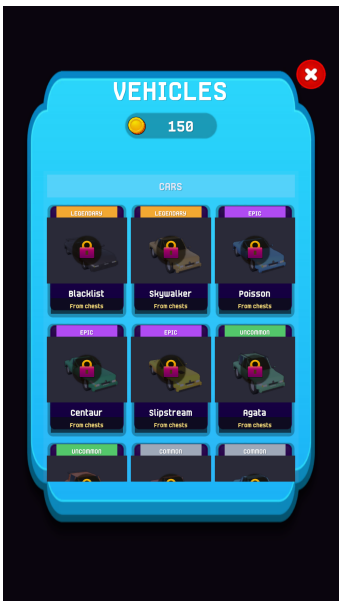
Main menu. The scene behind it is built at runtime by MenuBackground.cs.



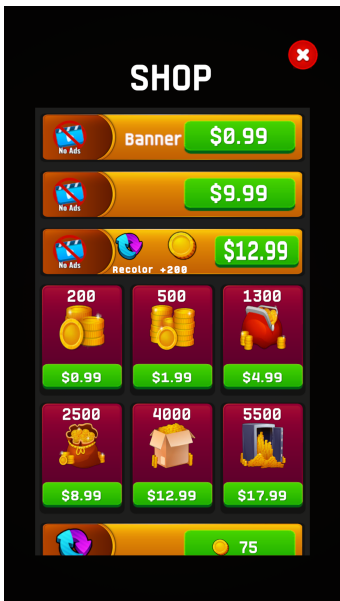
Gameplay: the jam grid, the parking row and the passenger queue.



Garage: chests, crafting and the shard economy.



The vehicle collection. Every card here is one of our generated models.



Store. Nine consumable tiers plus three non-consumables.

7. Third-party components we do use

We would rather list these than have them found. None of them is a game template, and none of them carries game logic. They are the ordinary building blocks of a Unity mobile title: the vendor SDKs Apple and Google require, and a small number of licensed art packs used for background dressing.

SDKs and engine packages

Component	Used for
Unity 6000.4.9f1 + Universal Render Pipeline	The engine itself
Firebase (Analytics, Messaging, Remote Config)	Analytics, push, remote config
Google Mobile Ads + UMP	Banner, interstitial and rewarded ads; consent
Unity In-App Purchasing	Store purchases
TextMesh Pro	Text rendering
Unity Input System	Touch input

Licensed art assets still present in the build

Pack	Files	What it is used for
300Mind — 2D Game UI Kit	13	Button, panel and frame sprites in the interface skin
SimplePoly City	104	Background buildings and street props behind the play area
Unity Technologies sample vehicles	12	One retained van model (“Connect”) plus traffic dressing
Low Poly Cars — Mega Pack	26	No longer referenced after the vehicle replacement
YelScriptFireStudio	15	Environment props
Polygonal Particles	4	Particle effects
Eugatnom	4	Environment props

All of these are ordinary paid or free Unity Asset Store licences held by our account. They are decoration around the game, not the game.

What we do *not* use

- No purchased or licensed game template, complete project, or “reskin kit.”
- No app generation service.
- No source code from another developer’s app.
- No shared codebase with any other app on the App Store. This is the only game we have submitted under this bundle identifier, and its code is not shared with our other project.

8. What we changed after the 26 August review

We took the rejection at face value: if the build reads as similar to other builds, then the parts that read that way have to go. These are the changes made on 26–27 August, all of them visible in commits `c0eafcb` and `e0339d8`.

Change	Detail
Passenger characters replaced	The free Asset Store character pack — the closest visual match to other games in the genre — was deleted and replaced with 20 characters we modelled ourselves.
Vehicle fleet replaced	27 of 28 garage vehicles are now generated by our own Blender tool. Vehicle art in the build dropped from 35 MB of third-party meshes to 10 MB of ours.
App renamed	“Bus Jam Traffic Rush” → Ridebury . The rename goes all the way down: the C# namespace is <code>Ridebury</code> , the main type is <code>RideburyGame</code> , and the editor menu root is <i>Ridebury</i> . Unity’s IL2CPP bakes type names into the binary, so this is a change to the binary, not just to the listing.
Unused stock assets purged	2,672 files removed. Tracked art in the repository went from roughly 293 MB to 135 MB. We verified that none of the 1,242 deleted asset GUIDs was referenced by any surviving asset.
Interface layout reworked	The in-game HUD was rebuilt away from the genre-standard arrangement: boosters moved from a bottom-centre row to a right-edge rail, settings moved to the top left, the coin bar to the top right, and the level badge was repositioned.
Identifiers deliberately kept	The bundle ID, the Firebase project and the Android signing key still carry the old name. Changing them would orphan existing installs and break Firebase, so we left them alone on purpose. We mention it here so the mismatch is not read as something hidden.

9. How this can be verified

None of the claims in this document rests on our word. Each one can be checked against artefacts we can hand over.

Claim	How to check it
The project was built from scratch over three months	Read access to the private repository. 144 commits, each with an author, a timestamp and a diff. The first commit is an empty Unity project.
The code is ours	89 C# files, 19,040 lines, growing commit by commit. The growth of the main controller from 650 to 5,056 lines is on page 4 and is reproducible with <code>git show</code> .
The characters are ours	<code>blender/bus_queue_characters.blend</code> is committed. It opens in Blender and contains all twenty models with their construction history, not an import.
The vehicles are ours	<code>blender/vehicles.blend</code> plus the 673-line Python generator (<code>vehLib.py</code> , <code>fleet.py</code>), which reproduces the whole fleet from scratch on any Blender install.
The remediation happened	Commits <code>c0eafc</code> and <code>e0339d8</code> , both dated 27 August 2026, with full diffs.

Contact

Oğuz Tarhan — Intake Entertainment

We can give the review team read access to the repository, send the Blender project files, or screen-share a walkthrough of the codebase at any time that suits you. Please reply in App Store Connect and we will arrange whichever is easiest.

Appendix files supplied alongside this document, in `docs/evidence/`: the build screenshots, seven Blender render sheets, the Blender workspace capture, and the two generator sources (`vehLib.py`, `fleet.py`).